

ROS2 Triager v2 — Master Cursor/Copilot Prompt

Paste this entire file into Cursor Composer (Ctrl+I) or Copilot Chat

ROLE & GOAL

You are implementing a complete upgrade of the `ros2-triager` ROS2 diagnostic tool. The goal: make it the best ROS2 diagnostic TUI in existence — accurate detection, zero-config auto-discovery, and a Textual-based TUI that surpasses `ros2_diag_tui`.

Two source specs have been merged. Follow this document exactly.

- Manus spec: preserve existing `checks/` folder, add correlation + odom/map
 - Claude spec: Textual TUI, typed StateBus, accurate detection logic, health scorer
-

ABSOLUTE RULES (never break these)

1. Use `rcipy` for ALL ROS2 calls — NEVER subprocess, NEVER `ros2 topic echo` CLI
 2. Use `Textual` for TUI — NEVER `curses`, NEVER `rich` interactive mode
 3. All state writes MUST acquire `bus._lock` (`threading.RLock`)
 4. Dead topic = `(publisher_count == 0)` OR `(age > threshold AND hz == 0.0)` — NOT just `hz==0`
 5. Keep ALL existing files in `checks/` — do not delete or break them
 6. Zero-config: if no YAML provided, auto-discover everything from live DDS graph
 7. Use `pydantic` BaseModel for all config schemas
 8. Type-hint everything — Python 3.10+ style (`(X | None)` not `Optional[X]`)
-

FINAL FILE STRUCTURE

```
ros2-triager/
├─ CURSOR_PROMPT.md      ← this file
├─ ARCHITECTURE.md       ← update after implementation
├─ README.md             ← update after implementation
├─ requirements.txt       ← add: textual, pydantic, networkx, pyyaml
├─ src/
│   └─ ros2_triage/
│       └─
```

- └─ command/
 - └─ triage.py ← MODIFY: add --tui flag, --preflight flag
- └─ checks/ ← KEEP ALL EXISTING FILES UNTOUCHED
 - └─ __init__.py
 - └─ finding.py ← keep as-is
 - └─ graph_utils.py ← MODIFY: expand InspectorNode (see §3.1)
 - └─ dead_topic.py ← MODIFY: fix detection logic (see §3.2)
 - └─ qos_check.py ← keep as-is
 - └─ tf_check.py ← keep as-is
 - └─ hz_check.py ← MODIFY: add deviation % (see §3.3)
 - └─ node_check.py ← keep as-is
 - └─ snapshot.py ← keep as-is
 - └─ latency_engine.py ← keep as-is
 - └─ dds_probe.py ← keep as-is
- └─ state/ ← NEW
 - └─ __init__.py
 - └─ state_bus.py ← NEW: thread-safe typed state store
- └─ collectors/ ← NEW
 - └─ __init__.py
 - └─ base_collector.py
 - └─ topic_collector.py
 - └─ tf_collector.py
 - └─ node_collector.py
 - └─ diagnostic_collector.py
 - └─ lifecycle_collector.py
 - └─ odom_map_collector.py ← NEW (from Manus)
- └─ engine/ ← NEW
 - └─ __init__.py
 - └─ dead_topic_detector.py
 - └─ zombie_node_detector.py
 - └─ tf_staleness_checker.py
 - └─ hz_deviation_checker.py
 - └─ diagnostics_parser.py
 - └─ lifecycle_checker.py
 - └─ health_scorer.py
- └─ ros_interface/ ← NEW (from Manus)
 - └─ __init__.py
 - └─ ros_node_manager.py
 - └─ message_converters.py
- └─ tui/ ← NEW (Textual, NOT curses)
 - └─ __init__.py

```

├── app.py
├── theme.tcss
├── widgets/
│   ├── __init__.py
│   ├── health_banner.py
│   ├── topics_table.py
│   ├── nodes_table.py
│   ├── tf_tree.py
│   ├── diagnostics_panel.py
│   ├── lifecycle_panel.py
│   ├── alerts_log.py
│   ├── graph_panel.py
│   ├── odom_map_panel.py  ← NEW (from Manus)
│   ├── inspect_panel.py
│   ├── sparkline.py
│   └── preflight_panel.py
├── screens/
│   ├── __init__.py
│   ├── main_screen.py
│   ├── help_screen.py
│   └── config_screen.py
├── export/                  ← NEW
│   ├── __init__.py
│   ├── snapshot_exporter.py
│   ├── csv_exporter.py
│   └── alert_webhook.py
├── config/                  ← NEW
│   ├── __init__.py
│   ├── config_manager.py
│   └── schema.py
├── correlation_engine.py ← MODIFY: wire to StateBus (from Manus)
├── reporter.py             ← keep as-is
├── interactive_tui.py      ← DEPRECATE (keep file, add deprecation warning)
├── config_manager.py       ← REPLACE with config/ package
├── expected_nodes.yaml
├── package.xml
├── setup.cfg
├── setup.py
└── test/
    ├── test_correlation_engine.py
    ├── test_dds_probe.py
    ├── test_dead_topic.py  ← UPDATE for new detection logic
    └── test_finding.py

```

```
|— test_latency_engine.py
|— test_qos_check.py
|— test_reporter.py
|— test_health_scorer.py    ← NEW
└— test_state_bus.py       ← NEW
```

§1 — STATE BUS `state/state_bus.py`

```
python
```

```

# state/state_bus.py
from __future__ import annotations
import threading
import time
from collections import deque
from dataclasses import dataclass, field
from typing import Any

@dataclass
class TopicState:
    name: str
    msg_type: str
    actual_hz: float = 0.0
    expected_hz: float | None = None
    bandwidth_kbps: float = 0.0
    publisher_count: int = 0
    subscriber_count: int = 0
    last_msg_time: float = 0.0          # monotonic
    status: str = "UNKNOWN"            # OK | LOW_HZ | DEAD | NO_PUB | STALE
    hz_history: deque = field(default_factory=lambda: deque(maxlen=60))
    is_critical: bool = False

@dataclass
class NodeState:
    name: str
    namespace: str = "/"
    pub_topics: list[str] = field(default_factory=list)
    sub_topics: list[str] = field(default_factory=list)
    services: list[str] = field(default_factory=list)
    is_zombie: bool = False
    status: str = "OK"                  # OK | ZOMBIE | MISSING

@dataclass
class TFState:
    frames: dict[str, str] = field(default_factory=dict)  # child → parent
    stale_frames: list[str] = field(default_factory=list)
    broken_chains: list[str] = field(default_factory=list)
    last_update: float = 0.0

@dataclass
class DiagItem:
    hardware_id: str

```

```
name: str
level: int # 0=OK 1=WARN 2=ERROR 3=STALE
message: str
values: dict[str, str] = field(default_factory=dict)
stamp: float = 0.0
```

```
@dataclass
```

```
class DiagState:
```

```
    items: dict[str, DiagItem] = field(default_factory=dict)
    error_count: int = 0
    warn_count: int = 0
```

```
@dataclass
```

```
class LifecycleState:
```

```
    name: str
    state_id: int = 0 # 1=UNCONFIGURED 2=INACTIVE 3=ACTIVE 4=FINALI
    state_label: str = "UNKNOWN"
    last_transition: float = 0.0
```

```
@dataclass
```

```
class OdomState:
```

```
    x: float = 0.0
    y: float = 0.0
    z: float = 0.0
    qx: float = 0.0
    qy: float = 0.0
    qz: float = 0.0
    qw: float = 1.0
    linear_vel: float = 0.0
    angular_vel: float = 0.0
    last_update: float = 0.0
```

```
@dataclass
```

```
class Alert:
```

```
    timestamp: float
    severity: str # ERROR | WARN | INFO
    source: str
    message: str
```

```
@dataclass
```

```
class HealthScore:
```

```
    topics_score: int = 100
```

```
nodes_score: int = 100
tf_score: int = 100
diag_score: int = 100
lifecycle_score: int = 100
overall: int = 100
```

```
@property
def color(self) -> str:
    if self.overall >= 80: return "green"
    if self.overall >= 50: return "yellow"
    return "red"

@property
def label(self) -> str:
    if self.overall >= 80: return "HEALTHY"
    if self.overall >= 50: return "DEGRADED"
    return "CRITICAL"
```

```
class StateBus:
```

```
    """Thread-safe shared state store. All collectors write here; TUI reads here."""
```

```
def __init__(self):
    self._lock = threading.RLock()
    self.topics: dict[str, TopicState] = {}
    self.nodes: dict[str, NodeState] = {}
    self.tf = TFState()
    self.diag = DiagState()
    self.lifecycle: dict[str, LifecycleState] = {}
    self.odom = OdomState()
    self.alerts: deque[Alert] = deque(maxlen=500)
    self.health = HealthScore()
    self._started_at: float = time.monotonic()

def push_alert(self, severity: str, source: str, message: str) -> None:
    with self._lock:
        self.alerts.appendleft(Alert(
            timestamp=time.time(),
            severity=severity,
            source=source,
            message=message,
        ))

def uptime_seconds(self) -> float:
    return time.monotonic() - self._started_at
```

§2 — BASE COLLECTOR `collectors/base_collector.py`

python


```
# collectors/base_collector.py
from __future__ import annotations
import threading
import time
from abc import ABC, abstractmethod
from rclpy.node import Node
from ..state.state_bus import StateBus

class BaseCollector(ABC):
    """
    Abstract base. Each subclass runs in a daemon thread,
    collecting data and writing to StateBus under bus._lock.
    """

    def __init__(self, node: Node, bus: StateBus, interval_sec: float = 1.0):
        self._node = node
        self._bus = bus
        self._interval = interval_sec
        self._running = False
        self._thread: threading.Thread | None = None

    def start(self) -> None:
        self._running = True
        self._thread = threading.Thread(target=self._loop, daemon=True)
        self._thread.start()

    def stop(self) -> None:
        self._running = False

    def _loop(self) -> None:
        while self._running:
            try:
                self.collect()
            except Exception as e:
                self._node.get_logger().warn(f"{self.__class__.__name__} error: {e}")
                time.sleep(self._interval)

    @abstractmethod
    def collect(self) -> None:
        """Override: read from ROS2 graph, write results to self._bus under self._bus._lock.
        """
        ...
```

§3 — KEY COLLECTORS

§3.1 `collectors/topic_collector.py`

python

```

# collectors/topic_collector.py
from __future__ import annotations
import time
from collections import deque
from rclpy.node import Node
from ..state.state_bus import StateBus, TopicState
from .base_collector import BaseCollector

class TopicCollector(BaseCollector):
    """
    Auto-discovers ALL topics via get_topic_names_and_types().
    Measures actual Hz using rolling deque of message timestamps.
    Measures bandwidth using serialized message size.
    """

    def __init__(self, node: Node, bus: StateBus):
        super().__init__(node, bus, interval_sec=1.0)
        self._timestamps: dict[str, deque] = {}  # topic → deque of float
        self._bytes: dict[str, int] = {}
        self._subscriptions: dict[str, object] = {}
        self._known_topics: set[str] = set()

    def collect(self) -> None:
        # Discover new topics
        current = {
            name: types[0] if types else "unknown"
            for name, types in self._node.get_topic_names_and_types()
        }

        for topic, msg_type in current.items():
            if topic not in self._known_topics:
                self._known_topics.add(topic)
                self._timestamps[topic] = deque(maxlen=100)
                self._bytes[topic] = 0
                self._subscribe(topic, msg_type)

        # Update state for all known topics
        with self._bus._lock:
            for topic in self._known_topics:
                ts = self._timestamps.get(topic, deque())
                hz = self._calc_hz(ts)
                pub_count = self._node.count_publishers(topic)
                sub_count = self._node.count_subscribers(topic)
                last_time = ts[-1] if ts else 0.0
                age = time.monotonic() - last_time if last_time else 9999.0

```

```

# Accurate status – see §DEAD TOPIC RULE
if pub_count == 0:
    status = "NO_PUB"
elif hz == 0.0 and age > 5.0:
    status = "DEAD"
elif hz > 0:
    existing = self._bus.topics.get(topic)
    exp = existing.expected_hz if existing else None
    if exp and abs(hz - exp) / exp > 0.5:
        status = "LOW_HZ"
    else:
        status = "OK"
else:
    status = "OK"

existing = self._bus.topics.get(topic)
bw = self._calc_bw(topic, ts)

if existing:
    existing.actual_hz = hz
    existing.publisher_count = pub_count
    existing.subscriber_count = sub_count
    existing.last_msg_time = last_time
    existing.status = status
    existing.bandwidth_kbps = bw
    existing.hz_history.append(hz)
else:
    self._bus.topics[topic] = TopicState(
        name=topic,
        msg_type=current.get(topic, "unknown"),
        actual_hz=hz,
        publisher_count=pub_count,
        subscriber_count=sub_count,
        last_msg_time=last_time,
        status=status,
        bandwidth_kbps=bw,
    )

def _subscribe(self, topic: str, msg_type_str: str) -> None:
    """Create a generic subscription using AnyMsg to track timestamps."""
    try:
        from rclpy.serialization import deserialize_message
        from rosidl_runtime_py.utilities import get_message
        import rclpy.qos as qos

        MsgType = get_message(msg_type_str)

```

```

qos_profile = qos.qos_profile_sensor_data

def cb(msg, t=topic):
    now = time.monotonic()
    self._timestamps[t].append(now)
    self._bytes[t] = self._bytes.get(t, 0) + len(str(msg))

    sub = self._node.create_subscription(MsgType, topic, cb, qos_profile)
    self._subscriptions[topic] = sub
except Exception:
    pass # Unknown type - will still track pub/sub counts

def _calc_hz(self, ts: deque) -> float:
    if len(ts) < 2:
        return 0.0
    window = min(len(ts), 10)
    recent = list(ts)[-window:]
    elapsed = recent[-1] - recent[0]
    return (len(recent) - 1) / elapsed if elapsed > 0 else 0.0

def _calc_bw(self, topic: str, ts: deque) -> float:
    if len(ts) < 2:
        return 0.0
    elapsed = ts[-1] - ts[0]
    if elapsed <= 0:
        return 0.0
    return (self._bytes.get(topic, 0) / elapsed) / 1024.0 # KB/s

```

§3.2 collectors/tf_collector.py

python

```

# collectors/tf_collector.py
from __future__ import annotations
import time
from rclpy.node import Node
from ..state.state_bus import StateBus, TFState
from .base_collector import BaseCollector

class TFCollector(BaseCollector):

    def __init__(self, node: Node, bus: StateBus):
        super().__init__(node, bus, interval_sec=0.5)
        self._tf_buffer = None
        self._tf_listener = None
        self._init_tf()

    def _init_tf(self) -> None:
        try:
            import tf2_ros
            self._tf_buffer = tf2_ros.Buffer()
            self._tf_listener = tf2_ros.TransformListener(self._tf_buffer, self._node)
        except ImportError:
            self._node.get_logger().warn("tf2_ros not available - TF checks disabled")

    def collect(self) -> None:
        if not self._tf_buffer:
            return

        try:
            raw = self._tf_buffer.all_frames_as_yaml()
            frames, stale = self._parse_frames(raw)

            with self._bus._lock:
                self._bus.tf = TFState(
                    frames=frames,
                    stale_frames=stale,
                    broken_chains=self._find_broken_chains(frames),
                    last_update=time.monotonic(),
                )
        except Exception as e:
            self._node.get_logger().debug(f"TFCollector: {e}")

    def _parse_frames(self, yaml_str: str) -> tuple[dict[str, str], list[str]]:
        """Parse tf2 YAML output into child→parent dict + stale list."""
        import yaml, time as t
        frames: dict[str, str] = {}

```

```

stale: list[str] = []
try:
    data = yaml.safe_load(yaml_str) or {}
    for frame_id, info in data.items():
        parent = info.get("parent", "")
        frames[frame_id] = parent
        age = info.get("most_recent_transform", 0.0)
        if age > 0.5:
            stale.append(frame_id)
except Exception:
    pass
return frames, stale

def _find_broken_chains(self, frames: dict[str, str]) -> list[str]:
    """Find frames whose parent chain never reaches a root."""
    roots = {f for f, p in frames.items() if not p or p not in frames}
    broken = []
    for frame in frames:
        visited = set()
        cur = frame
        while cur in frames and cur not in roots:
            if cur in visited:
                broken.append(frame)
                break
            visited.add(cur)
            cur = frames[cur]
    return broken

```

§3.3 collectors/diagnostic_collector.py

python

```

# collectors/diagnostic_collector.py
from __future__ import annotations
import time
import rclpy.qos as qos
from diagnostic_msgs.msg import DiagnosticArray, DiagnosticStatus
from rclpy.node import Node
from ..state.state_bus import StateBus, DiagItem, DiagState
from .base_collector import BaseCollector

class DiagnosticCollector(BaseCollector):
    """
    Subscribes to /diagnostics AND /diagnostics_agg.
    Converts DiagnosticStatus into DiagItem in StateBus.
    Also converts to Finding objects for compatibility with existing checks/.
    """

    def __init__(self, node: Node, bus: StateBus):
        super().__init__(node, bus, interval_sec=99999) # event-driven
        self._sub = None
        self._sub_agg = None

    def start(self) -> None:
        self._sub = self._node.create_subscription(
            DiagnosticArray, "/diagnostics",
            self._callback, qos.qos_profile_sensor_data
        )
        try:
            self._sub_agg = self._node.create_subscription(
                DiagnosticArray, "/diagnostics_agg",
                self._callback, qos.qos_profile_sensor_data
            )
        except Exception:
            pass

    def collect(self) -> None:
        pass # callback-driven

    def _callback(self, msg: DiagnosticArray) -> None:
        items: dict[str, DiagItem] = {}
        errors = warns = 0

        for status in msg.status:
            key = f"{status.hardware_id}/{status.name}"
            level = int(status.level)
            if level >= DiagnosticStatus.ERROR:

```



```
        errors += 1
    elif level == DiagnosticStatus.WARN:
        warns += 1

    items[key] = DiagItem(
        hardware_id=status.hardware_id,
        name=status.name,
        level=level,
        message=status.message,
        values={kv.key: kv.value for kv in status.values},
        stamp=time.time(),
    )

    with self._bus._lock:
        self._bus.diag = DiagState(
            items=**self._bus.diag.items, **items,
            error_count=errors,
            warn_count=warns,
        )
```

§3.4 `collectors/lifecycle_collector.py`

python

```

# collectors/lifecycle_collector.py
from __future__ import annotations
import time
from rclpy.node import Node
from ..state.state_bus import StateBus, LifecycleState
from .base_collector import BaseCollector

LIFECYCLE_LABELS = {1: "UNCONFIGURED", 2: "INACTIVE", 3: "ACTIVE", 4: "FINALIZED"}

class LifecycleCollector(BaseCollector):

    def __init__(self, node: Node, bus: StateBus):
        super().__init__(node, bus, interval_sec=2.0)

    def collect(self) -> None:
        # Discover lifecycle nodes by checking for /get_state service
        all_nodes = self._node.get_node_names_and_namespaces()
        lifecycle_nodes = []

        for name, ns in all_nodes:
            full = f"{ns}/{name}".replace("//", "/")
            svc = f"{full}/get_state"
            if svc in [s for s, _ in self._node.get_service_names_and_types()]:
                lifecycle_nodes.append((name, ns, full))

        updated = {}
        for name, ns, full in lifecycle_nodes:
            try:
                from lifecycle_msgs.srv import GetState
                cli = self._node.create_client(GetState, f"{full}/get_state")
                if cli.wait_for_service(timeout_sec=0.3):
                    future = cli.call_async(GetState.Request())
                    import rclpy
                    rclpy.spin_until_future_complete(self._node, future, timeout_sec=None)
                    if future.result():
                        sid = future.result().current_state.id
                        updated[full] = LifecycleState(
                            name=full,
                            state_id=sid,
                            state_label=LIFECYCLE_LABELS.get(sid, "UNKNOWN"),
                            last_transition=time.monotonic(),
                        )
                    self._node.destroy_client(cli)
            except Exception:
                pass

```

```
with self._bus._lock:  
    self._bus.lifecycle.update(updated)
```

§3.5 `collectors/odom_map_collector.py` (*from Manus*)

python

```

# collectors/odom_map_collector.py
from __future__ import annotations
import time
from rclpy.node import Node
from nav_msgs.msg import Odometry, OccupancyGrid
from ..state.state_bus import StateBus, OdomState
from .base_collector import BaseCollector
import rclpy.qos as qos

class OdomMapCollector(BaseCollector):
    """Subscribes to /odom and /map for TUI visualization."""

    def __init__(self, node: Node, bus: StateBus):
        super().__init__(node, bus, interval_sec=99999) # callback-driven
        self._map_data: list | None = None
        self._map_info = None

    def start(self) -> None:
        self._odom_sub = self._node.create_subscription(
            Odometry, "/odom", self._odom_cb, qos.qos_profile_sensor_data
        )
        self._map_sub = self._node.create_subscription(
            OccupancyGrid, "/map", self._map_cb,
            rclpy.qos.QoSProfile(
                depth=1,
                durability=rclpy.qos.QoSDurabilityPolicy.TRANSIENT_LOCAL,
                reliability=rclpy.qos.QoSReliabilityPolicy.RELIABLE,
            )
        )

    def collect(self) -> None:
        pass # callback-driven

    def _odom_cb(self, msg: Odometry) -> None:
        with self._bus._lock:
            p = msg.pose.pose.position
            o = msg.pose.pose.orientation
            t = msg.twist.twist
            self._bus.odom = OdomState(
                x=p.x, y=p.y, z=p.z,
                qx=o.x, qy=o.y, qz=o.z, qw=o.w,
                linear_vel=(t.linear.x**2 + t.linear.y**2)**0.5,
                angular_vel=t.angular.z,
                last_update=time.monotonic(),
            )

```

```
def _map_cb(self, msg: OccupancyGrid) -> None:
    # Store map metadata for TUI visualization
    self._map_data = list(msg.data)
    self._map_info = {
        "width": msg.info.width,
        "height": msg.info.height,
        "resolution": msg.info.resolution,
        "origin_x": msg.info.origin.position.x,
        "origin_y": msg.info.origin.position.y,
    }
```

§4 — ANALYSIS ENGINE

§4.1 `engine/dead_topic_detector.py`

python

```

# engine/dead_topic_detector.py
# DEAD TOPIC RULE – must satisfy this exact logic (not just hz==0):
#
# DEAD    = publisher_count == 0 AND age > threshold
# NO_PUB  = publisher_count == 0 (regardless of age)
# LOW_HZ  = hz > 0 AND |hz - expected| / expected > 0.5
# OK      = everything else
#
# Rationale: latched topics (/map, /tf_static) publish once, then hz=0 forever.
# Checking publisher_count avoids false-positives on these topics.

import time
from ..state.state_bus import TopicState

def classify_topic(topic: TopicState, dead_threshold_sec: float = 5.0) -> str:
    age = time.monotonic() - topic.last_msg_time if topic.last_msg_time else 9999.0

    if topic.publisher_count == 0:
        if age > dead_threshold_sec:
            return "DEAD"
        return "NO_PUB"

    if topic.actual_hz == 0.0 and age > dead_threshold_sec:
        return "DEAD"

    if topic.expected_hz and topic.actual_hz > 0:
        deviation = abs(topic.actual_hz - topic.expected_hz) / topic.expected_hz
        if deviation > 0.5:
            return "LOW_HZ"
        if deviation > 0.2:
            return "WARN_HZ"

    return "OK"

```

§4.2 engine/zombie_node_detector.py

python

```
# engine/zombie_node_detector.py
from ..state.state_bus import NodeState, TopicState

def is_zombie(node: NodeState, topics: dict[str, TopicState]) -> bool:
    """
    Node is a zombie if:
    - It has at least one published topic
    - ALL of its published topics are DEAD or NO_PUB
    """
    if not node.pub_topics:
        return False
    return all(
        topics.get(t, TopicState(name=t, msg_type="")).status in ("DEAD", "NO_PUB")
        for t in node.pub_topics
    )
```

§4.3 engine/health_scorer.py

python

```

# engine/health_scorer.py
from ..state.state_bus import StateBus, HealthScore

def compute_health(bus: StateBus) -> HealthScore:
    with bus._lock:
        topics = list(bus.topics.values())
        nodes = list(bus.nodes.values())
        tf = bus.tf
        diag = bus.diag
        lifecycle = list(bus.lifecycle.values())

    # Topics score: -20 per DEAD critical, -10 per DEAD non-critical, -5 per LOW_HZ
    t_score = 100
    for t in topics:
        if t.status == "DEAD":
            t_score -= 20 if t.is_critical else 10
        elif t.status in ("LOW_HZ", "NO_PUB"):
            t_score -= 5

    # Nodes score: -30 per zombie
    n_score = 100 - sum(30 for n in nodes if n.is_zombie)

    # TF score: -15 per stale frame, -30 per broken chain
    tf_score = 100 - len(tf.stale_frames) * 15 - len(tf.broken_chains) * 30

    # Diagnostics score: -20 per ERROR, -5 per WARN
    d_score = 100 - diag.error_count * 20 - diag.warn_count * 5

    # Lifecycle score: -20 per UNCONFIGURED, -10 per INACTIVE
    lc_score = 100
    for lc in lifecycle:
        if lc.state_id == 1: lc_score -= 20
        elif lc.state_id == 2: lc_score -= 10

    def clamp(v: int) -> int:
        return max(0, min(100, v))

    scores = [clamp(t_score), clamp(n_score), clamp(tf_score),
              clamp(d_score), clamp(lc_score)]

    # Weighted average: topics 30%, nodes 20%, tf 20%, diag 20%, lifecycle 10%
    weights = [0.30, 0.20, 0.20, 0.20, 0.10]
    overall = int(sum(s * w for s, w in zip(scores, weights)))

    return HealthScore(

```



```
    topics_score=clamp(t_score),  
    nodes_score=clamp(n_score),  
    tf_score=clamp(tf_score),  
    diag_score=clamp(d_score),  
    lifecycle_score=clamp(lc_score),  
    overall=overall,  
)
```

§5 — TUI (Textual — NOT curses)

§5.1 `tui/app.py`

```
python
```

```
# tui/app.py
from __future__ import annotations
import rclpy
import threading
from textual.app import App, ComposeResult
from textual.binding import Binding
from textual.widgets import Header, Footer, TabbedContent, TabPane

from ..state.state_bus import StateBus
from ..collectors.topic_collector import TopicCollector
from ..collectors.tf_collector import TFCollector
from ..collectors.node_collector import NodeCollector
from ..collectors.diagnostic_collector import DiagnosticCollector
from ..collectors.lifecycle_collector import LifecycleCollector
from ..collectors.odom_map_collector import OdomMapCollector
from ..engine.health_scorer import compute_health
from ..engine.dead_topic_detector import classify_topic
from ..engine.zombie_node_detector import is_zombie

from .widgets.health_banner import HealthBanner
from .widgets.topics_table import TopicsTable
from .widgets.nodes_table import NodesTable
from .widgets.tf_tree import TFTreeWidget
from .widgets.diagnostics_panel import DiagnosticsPanel
from .widgets.lifecycle_panel import LifecyclePanel
from .widgets.alerts_log import AlertsLog
from .widgets.graph_panel import GraphPanel
from .widgets.odom_map_panel import OdomMapPanel
from .widgets.inspect_panel import InspectPanel
from .widgets.preflight_panel import PreflightPanel

class TriagerApp(App):
    CSS_PATH = "theme.tcss"

    BINDINGS = [
        Binding("q", "quit", "Quit"),
        Binding("h", "push_screen('help')", "Help"),
        Binding("s", "snapshot", "Snapshot"),
        Binding("r", "reload", "Reload"),
        Binding("p", "preflight", "Preflight"),
        Binding("f", "filter", "Filter"),
        Binding("a", "toggle_alerts", "Alerts"),
        Binding("1", "switch_tab('topics')", "Topics"),
        Binding("2", "switch_tab('nodes')", "Nodes"),
        Binding("3", "switch_tab('tf')", "TF"),
```

```

Binding("4", "switch_tab('diagnostics')", "Diagnostics"),
Binding("5", "switch_tab('lifecycle')", "Lifecycle"),
Binding("6", "switch_tab('graph')", "Graph"),
Binding("7", "switch_tab('odom')", "Odom/Map"),
Binding("8", "switch_tab('alerts')", "Alerts"),
Binding("9", "switch_tab('preflight')", "Preflight"),
]

def __init__(self, bus: StateBus, config: dict | None = None):
    super().__init__()
    self.bus = bus
    self.config = config or {}

def compose(self) -> ComposeResult:
    yield Header()
    yield HealthBanner(self.bus)
    with TabbedContent(initial="topics"):
        with TabPane("Topics", id="topics"):
            yield TopicsTable(self.bus)
        with TabPane("Nodes", id="nodes"):
            yield NodesTable(self.bus)
        with TabPane("TF Tree", id="tf"):
            yield TFTreeWidget(self.bus)
        with TabPane("Diagnostics", id="diagnostics"):
            yield DiagnosticsPanel(self.bus)
        with TabPane("Lifecycle", id="lifecycle"):
            yield LifecyclePanel(self.bus)
        with TabPane("Graph", id="graph"):
            yield GraphPanel(self.bus)
        with TabPane("Odom/Map", id="odom"):
            yield OdomMapPanel(self.bus)
        with TabPane("Alerts", id="alerts"):
            yield AlertsLog(self.bus)
        with TabPane("Preflight", id="preflight"):
            yield PreflightPanel(self.bus, self.config)
    yield Footer()

def on_mount(self) -> None:
    # Refresh TUI every 1 second from bus state
    self.set_interval(1.0, self._refresh_all)

def _refresh_all(self) -> None:
    # Recompute health score
    score = compute_health(self.bus)
    with self.bus._lock:
        self.bus.health = score

```

```

# Reclassify all topics
with self.bus._lock:
    for t in self.bus.topics.values():
        t.status = classify_topic(t)

# Reclassify zombie nodes
with self.bus._lock:
    for n in self.bus.nodes.values():
        n.is_zombie = is_zombie(n, self.bus.topics)

# Notify all widgets to refresh
self.query_one(HealthBanner).refresh()
try:
    self.query_one(TopicsTable).refresh_data()
except Exception:
    pass

def action_snapshot(self) -> None:
    from ..export.snapshot_exporter import export_snapshot
    import json, datetime
    path = f"/tmp/ros2_triage_{datetime.datetime.now().strftime('%Y%m%d_%H%M%S')}
    with open(path, "w") as f:
        json.dump(export_snapshot(self.bus), f, indent=2, default=str)
    self.notify(f"Snapshot saved: {path}")

def action_switch_tab(self, tab_id: str) -> None:
    self.query_one(TabbedContent).active = tab_id

```

§5.2 tui/theme.tcss

CSS

```
/* theme.tcss - Textual CSS for ros2-triager */

Screen {
    background: $surface;
}

HealthBanner {
    height: 5;
    layout: horizontal;
    padding: 0 1;
}

HealthCard {
    border: tall $panel;
    background: $panel;
    height: 5;
    width: 1fr;
    padding: 0 1;
    margin: 0 1;
    content-align: center middle;
}

HealthCard.ok      { border: tall $success; }
HealthCard.degraded { border: tall $warning; }
HealthCard.critical { border: tall $error; }

DataTable {
    height: 1fr;
}

DataTable > .datatable--row-hover {
    background: $boost;
}

.status-ok      { color: $success; }
.status-warn    { color: $warning; }
.status-dead    { color: $error; text-style: bold; }
.status-stale   { color: $warning; text-style: italic; }
.status-zombie  { color: $error; text-style: bold italic; }

AlertsLog {
    height: 1fr;
    overflow-y: scroll;
}

.alert-error { background: $error 20%; }
```

```
.alert-warn { background: $warning 20%; }
```

```
TFTreeWidget Tree {  
    height: 1fr;  
}
```

```
.stale-frame { color: $warning; }  
.broken-frame { color: $error; }
```

```
PreflightPanel {  
    height: 1fr;  
    padding: 1 2;  
}
```

```
.preflight-pass { color: $success; }  
.preflight-fail { color: $error; }  
.preflight-pending { color: $warning; }
```

§5.3 `tui/widgets/health_banner.py`

```
python
```

```

# tui/widgets/health_banner.py
from textual.app import ComposeResult
from textual.widget import Widget
from textual.widgets import Static
from textual.reactive import reactive
from ..state.state_bus import StateBus

class HealthCard(Static):
    def __init__(self, label: str, value: str = "-", css_class: str = "ok"):
        super().__init__()
        self._label = label
        self._value = value
        self._css = css_class

    def render(self) -> str:
        return f"[dim]{self._label}[/]\\n[bold]{self._value}[/]"

    def update_value(self, value: str, css_class: str = "ok") -> None:
        self._value = value
        self._css = css_class
        self.remove_class("ok", "degraded", "critical")
        self.add_class(css_class)
        self.refresh()

class HealthBanner(Widget):

    def __init__(self, bus: StateBus):
        super().__init__()
        self.bus = bus

    def compose(self) -> ComposeResult:
        yield HealthCard("SYSTEM HEALTH", "-", "ok")
        yield HealthCard("TOPICS", "-")
        yield HealthCard("NODES", "-")
        yield HealthCard("TF FRAMES", "-")
        yield HealthCard("/diagnostics", "-")
        yield HealthCard("LIFECYCLE", "-")

    def refresh(self) -> None:
        with self.bus._lock:
            h = self.bus.health
            topics = list(self.bus.topics.values())
            nodes = list(self.bus.nodes.values())
            tf = self.bus.tf

```

```

        diag = self.bus.diag
        lc = list(self.bus.lifecycle.values())

cards = list(self.query(HealthCard))
if not cards:
    return

# Overall health
cards[0].update_value(f"{h.overall}/100 [{h.label}]", h.color)

# Topics
dead = sum(1 for t in topics if t.status == "DEAD")
cards[1].update_value(
    f"{len(topics)-dead}/{len(topics)}",
    "ok" if dead == 0 else "critical" if dead > 2 else "degraded"
)

# Nodes
zombies = sum(1 for n in nodes if n.is_zombie)
cards[2].update_value(
    f"{len(nodes)-zombies}/{len(nodes)}",
    "ok" if zombies == 0 else "critical"
)

# TF
stale = len(tf.stale_frames)
cards[3].update_value(
    f"{len(tf.frames)} frames",
    "ok" if stale == 0 else "degraded"
)

# Diagnostics
cards[4].update_value(
    f"{diag.error_count} ERR {diag.warn_count} WARN",
    "critical" if diag.error_count > 0 else "degraded" if diag.warn_count >
)

# Lifecycle
inactive = sum(1 for l in lc if l.state_id != 3)
cards[5].update_value(
    f"{len(lc)-inactive}/{len(lc)} ACTIVE",
    "ok" if inactive == 0 else "degraded"
)

```


§5.4 tui/widgets/topics_table.py

python

```
# tui/widgets/topics_table.py
from textual.app import ComposeResult
from textual.widget import Widget
from textual.widgets import DataTable, Input
from textual.reactive import reactive
from ..state.state_bus import StateBus

BRAILLE = " .....:||||"

def hz_sparkline(history: list, width: int = 8) -> str:
    if not history:
        return " " * width
    mx = max(history) or 1
    samples = list(history)[-width:]
    return "".join(
        BRAILLE[min(int(v / mx * (len(BRAILLE) - 1)), len(BRAILLE) - 1)]
        for v in samples
    )

STATUS_ICON = {
    "OK": "●", "LOW_HZ": "▲", "WARN_HZ": "▲",
    "DEAD": "✖", "NO_PUB": "✖", "STALE": "◌", "UNKNOWN": "?"
}

class TopicsTable(Widget):

    def __init__(self, bus: StateBus):
        super().__init__()
        self.bus = bus
        self._filter = ""

    def compose(self) -> ComposeResult:
        yield Input(placeholder="filter topics...", id="topic-filter")
        tbl = DataTable(id="topics-dt")
        tbl.add_columns("", "Topic", "Hz", "Exp", "BW KB/s", "Pub", "Trend", "Status")
        yield tbl

    def on_input_changed(self, event: Input.Changed) -> None:
        self._filter = event.value.lower()
        self.refresh_data()

    def refresh_data(self) -> None:
        tbl = self.query_one("#topics-dt", DataTable)
        tbl.clear()
```

```

with self.bus._lock:
    topics = sorted(
        self.bus.topics.values(),
        key=lambda t: (0 if t.status == "DEAD" else 1 if "WARN" in t.status
    )

for t in topics:
    if self._filter and self._filter not in t.name.lower():
        continue
    icon = STATUS_ICON.get(t.status, "?")
    exp = f"{t.expected_hz:.1f}" if t.expected_hz else "_"
    spark = hz_sparkline(list(t.hz_history))
    status_str = t.status
    tbl.add_row(
        icon,
        t.name,
        f"{t.actual_hz:.1f}",
        exp,
        f"{t.bandwidth_kbps:.1f}",
        str(t.publisher_count),
        spark,
        status_str,
        key=t.name,
    )

```

§6 — EXISTING CHECKS MODIFICATIONS

§6.1 Fix `checks/dead_topic.py` — update detection logic

```

python

# In checks/dead_topic.py — REPLACE the staleness check with:
#
# OLD (wrong — false positives on latched topics):
#   if topic_hz == 0.0:
#       yield Finding(...)
#
# NEW (correct):
#   from ros2_triage.engine.dead_topic_detector import classify_topic
#   status = classify_topic(topic_state)
#   if status in ("DEAD", "NO_PUB"):
#       yield Finding(check="dead_topic", severity=Severity.CRITICAL, ...)

```

§6.2 Enhance `checks/hz_check.py` — add deviation %

```
python

# In checks/hz_check.py — ADD deviation percentage to Finding message:
#
# OLD: message = f"Topic {name} publishing at {actual:.1f} Hz (expected {expected:.1f} Hz)"
# NEW:
#     deviation_pct = abs(actual - expected) / expected * 100
#     message = f"Topic {name}: {actual:.1f} Hz vs {expected:.1f} expected ({deviation_pct:.1f} % deviation)"
```

§6.3 Expand `checks/graph_utils.py` — InspectorNode

```
python

# In checks/graph_utils.py — ADD these methods to InspectorNode:

def get_all_topics_with_info(self) -> dict:
    """Returns {topic_name: {type, pub_count, sub_count}}"""
    result = {}
    for name, types in self.get_topic_names_and_types():
        result[name] = {
            "type": types[0] if types else "unknown",
            "pub_count": self.count_publishers(name),
            "sub_count": self.count_subscribers(name),
        }
    return result

def get_all_nodes_with_info(self) -> list[dict]:
    """Returns [{name, namespace, pub_topics, sub_topics, services}]"""
    result = []
    for name, ns in self.get_node_names_and_namespaces():
        try:
            pubs = [t for t, _ in self.get_publisher_names_and_types_by_node(name, ns)]
            subs = [t for t, _ in self.get_subscriber_names_and_types_by_node(name, ns)]
            svcs = [s for s, _ in self.get_service_names_and_types_by_node(name, ns)]
            result.append({"name": name, "namespace": ns,
                           "pub_topics": pubs, "sub_topics": subs, "services": svcs})
        except Exception:
            result.append({"name": name, "namespace": ns,
                           "pub_topics": [], "sub_topics": [], "services": []})
    return result
```

python

```

# correlation_engine.py – MODIFY existing to wire into StateBus
# ADD this method alongside existing correlation logic:

def correlate_from_bus(self, bus: "StateBus") -> list[dict]:
    """
    Run correlation analysis using live StateBus state.
    Returns list of hypothesis dicts: {confidence, root_cause, affected, suggestion}
    """
    with bus._lock:
        dead_topics = [t for t in bus.topics.values() if t.status == "DEAD"]
        zombie_nodes = [n for n in bus.nodes.values() if n.is_zombie]
        stale_tf = bus.tf.stale_frames
        diag_errors = [i for i in bus.diag.items.values() if i.level >= 2]

    hypotheses = []

    # Pattern: zombie node + dead topics → node crash likely
    if zombie_nodes and dead_topics:
        hypotheses.append({
            "confidence": 0.85,
            "root_cause": f"Node(s) {[n.name for n in zombie_nodes]} appear crashed",
            "affected": [t.name for t in dead_topics],
            "suggestion": "Check node logs: ros2 log or journalctl"
        })

    # Pattern: stale TF + dead odom → odometry pipeline broken
    if stale_tf and any(t.name == "/odom" for t in dead_topics):
        hypotheses.append({
            "confidence": 0.9,
            "root_cause": "Odometry pipeline broken – TF frames going stale",
            "affected": stale_tf,
            "suggestion": "Check wheel encoders and robot_state_publisher"
        })

    # Pattern: diagnostic errors → cross-reference with dead topics
    for err in diag_errors:
        hypotheses.append({
            "confidence": 0.7,
            "root_cause": f"Hardware issue: {err.hardware_id} – {err.message}",
            "affected": [err.name],
            "suggestion": "Check hardware driver and cable connections"
        })

    return sorted(hypotheses, key=lambda h: h["confidence"], reverse=True)

```

§8 — ENTRY POINT `command/triage.py` modifications

python

```
# command/triage.py - ADD to existing CLI:
```

```
def add_tui_subcommand(subparsers):
    tui_parser = subparsers.add_parser("tui", help="Launch interactive TUI dashboard")
    tui_parser.add_argument("--config", "-c", default=None, help="Path to YAML config")
    tui_parser.add_argument("--domain", "-d", type=int, default=0, help="ROS_DOMAIN_ID")
    tui_parser.add_argument("--preflight", action="store_true", help="Run preflight checks")
    tui_parser.add_argument("--preflight-timeout", type=int, default=30)
    return tui_parser

def run_tui(args):
    import os
    os.environ["ROS_DOMAIN_ID"] = str(args.domain)

    import rclpy
    from ..state.state_bus import StateBus
    from ..collectors.topic_collector import TopicCollector
    from ..collectors.tf_collector import TFCollector
    from ..collectors.node_collector import NodeCollector
    from ..collectors.diagnostics_collector import DiagnosticCollector
    from ..collectors.lifecycle_collector import LifecycleCollector
    from ..collectors.odom_map_collector import OdomMapCollector
    from ..config.config_manager import load_config
    from ..tui.app import TriagerApp

    rclpy.init()
    node = rclpy.create_node("_ros2_triager_node")
    bus = StateBus()
    config = load_config(args.config)

    # Start all collectors
    collectors = [
        TopicCollector(node, bus),
        TFCollector(node, bus),
        NodeCollector(node, bus),
        DiagnosticCollector(node, bus),
        LifecycleCollector(node, bus),
        OdomMapCollector(node, bus),
    ]
    for c in collectors:
        c.start()

    # Spin rclpy in background thread
    import threading
    spin_thread = threading.Thread(
```



```

        target=lambda: rclpy.spin(node), daemon=True
    )
    spin_thread.start()

    if args.preflight:
        _run_preflight(bus, config, args.preflight_timeout)
        return

    app = TriagerApp(bus, config)
    app.run()

    # Cleanup
    for c in collectors:
        c.stop()
    node.destroy_node()
    rclpy.shutdown()

def _run_preflight(bus, config, timeout_sec):
    """Run headless preflight check. Exit 0=pass, 1=fail."""
    import time, sys
    deadline = time.monotonic() + timeout_sec
    print(f"Running preflight check (timeout: {timeout_sec}s)...")

    while time.monotonic() < deadline:
        from ..engine.health_scorer import compute_health
        score = compute_health(bus)
        if score.overall >= 80:
            print(f"✓ PREFLIGHT PASSED – health score: {score.overall}/100")
            sys.exit(0)
        time.sleep(1.0)

    print(f"✗ PREFLIGHT FAILED – timeout after {timeout_sec}s")
    sys.exit(1)

```

§9 — CONFIG `config/schema.py` + `config/config_manager.py`

python

```

# config/schema.py
from pydantic import BaseModel, Field


class TopicConfig(BaseModel):
    name: str
    topic: str
    type: str | None = None
    expected_hz: float | None = None
    critical: bool = False
    dead_threshold_sec: float = 5.0


class TFFrameCheck(BaseModel):
    parent: str
    child: str
    description: str = ""


class Settings(BaseModel):
    refresh_interval_ms: int = 1000
    stale_data_timeout_sec: float = 5.0
    dead_topic_threshold_sec: float = 5.0
    hz_deviation_warn_pct: float = 20.0
    hz_deviation_dead_pct: float = 50.0
    preflight_min_health: int = 80


class TriagerConfig(BaseModel):
    settings: Settings = Field(default_factory=Settings)
    monitored_topics: list[TopicConfig] = Field(default_factory=list)
    tf_frames: list[TFFrameCheck] = Field(default_factory=list)
    lifecycle_nodes: list[str] = Field(default_factory=list)
    alert_webhook_url: str | None = None
    robot_name: str = "robot"


# config/config_manager.py
import yaml, os
from .schema import TriagerConfig


def load_config(path: str | None = None) -> dict:
    """Load YAML config. If no path, return defaults (zero-config mode)."""
    if path and os.path.exists(path):
        with open(path) as f:

```

```
        raw = yaml.safe_load(f) or {}
        cfg = TriagerConfig(**raw)
    else:
        cfg = TriagerConfig()
    return cfg.model_dump()
```

§10 — EXPORT `export/snapshot_exporter.py`

python

```
# export/snapshot_exporter.py
from __future__ import annotations
import dataclasses, time
from ..state.state_bus import StateBus

def export_snapshot(bus: StateBus) -> dict:
    with bus._lock:
        return {
            "timestamp": time.time(),
            "uptime_sec": bus.uptime_seconds(),
            "health": dataclasses.asdict(bus.health),
            "topics": {k: dataclasses.asdict(v) for k, v in bus.topics.items()},
            "nodes": {k: dataclasses.asdict(v) for k, v in bus.nodes.items()},
            "tf": dataclasses.asdict(bus.tf),
            "diagnostics": dataclasses.asdict(bus.diag),
            "lifecycle": {k: dataclasses.asdict(v) for k, v in bus.lifecycle.items()},
            "odometer": dataclasses.asdict(bus.odometer),
            "alerts": [dataclasses.asdict(a) for a in list(bus.alerts)[:50]],
        }
```

§11 — `requirements.txt`

```
textual>=0.50.0
pydantic>=2.0.0
networkx>=3.0
pyyaml>=6.0
```

§12 — `package.xml` additions

xml

```
<!-- ADD these to existing package.xml -->
<depend>rclpy</depend>
<depend>tf2_ros</depend>
<depend>diagnostic_msgs</depend>
<depend>lifecycle_msgs</depend>
<depend>sensor_msgs</depend>
<depend>nav_msgs</depend>
<exec_depend>python3-textual</exec_depend>
<exec_depend>python3-pydantic</exec_depend>
<exec_depend>python3-networkx</exec_depend>
```

§13 — IMPLEMENTATION ORDER FOR CURSOR

Implement files in this exact order to avoid import errors:

STEP 1: state/state_bus.py
STEP 2: collectors/base_collector.py
STEP 3: collectors/topic_collector.py
STEP 4: collectors/tf_collector.py
STEP 5: collectors/node_collector.py
STEP 6: collectors/diagnostic_collector.py
STEP 7: collectors/lifecycle_collector.py
STEP 8: collectors/odom_map_collector.py
STEP 9: engine/dead_topic_detector.py
STEP 10: engine/zombie_node_detector.py
STEP 11: engine/tf_staleness_checker.py
STEP 12: engine/hz_deviation_checker.py
STEP 13: engine/diagnostics_parser.py
STEP 14: engine/lifecycle_checker.py
STEP 15: engine/health_scorer.py
STEP 16: ros_interface/ros_node_manager.py
STEP 17: ros_interface/message_converters.py
STEP 18: config/schema.py
STEP 19: config/config_manager.py
STEP 20: export/snapshot_exporter.py
STEP 21: export/csv_exporter.py
STEP 22: export/alert_webhook.py
STEP 23: tui/theme.tcss
STEP 24: tui/widgets/health_banner.py
STEP 25: tui/widgets/sparkline.py
STEP 26: tui/widgets/topics_table.py
STEP 27: tui/widgets/nodes_table.py
STEP 28: tui/widgets/tf_tree.py
STEP 29: tui/widgets/diagnostics_panel.py
STEP 30: tui/widgets/lifecycle_panel.py
STEP 31: tui/widgets/odom_map_panel.py
STEP 32: tui/widgets/graph_panel.py
STEP 33: tui/widgets/alerts_log.py
STEP 34: tui/widgets/inspect_panel.py
STEP 35: tui/widgets/preflight_panel.py
STEP 36: tui/screens/main_screen.py
STEP 37: tui/screens/help_screen.py
STEP 38: tui/app.py
STEP 39: MODIFY checks/graph_utils.py (add methods from §6.3)
STEP 40: MODIFY checks/dead_topic.py (fix logic from §6.1)
STEP 41: MODIFY checks/hz_check.py (add deviation % from §6.2)
STEP 42: MODIFY correlation_engine.py (add correlate_from_bus from §7)
STEP 43: MODIFY command/triage.py (add TUI subcommand from §8)
STEP 44: test/test_health_scorer.py
STEP 45: test/test_state_bus.py
STEP 46: test/test_dead_topic.py (update for new logic)

```
STEP 47: UPDATE requirements.txt
STEP 48: UPDATE package.xml
```

§14 — KEYBOARD SHORTCUTS (complete reference)

Key	Action
q	Quit
h	Help overlay
1-9	Switch tab directly
Tab / Shift+Tab	Cycle panel focus
↑ ↓	Navigate rows
Enter	Inspect selected
f	Toggle filter bar
s	Export JSON snapshot
r	Reload config + rediscover
d	Change ROS_DOMAIN_ID
p	Run preflight check
a	Toggle alert log
+ / -	Faster / slower refresh

§15 — HOW TO RUN

```
bash
```

```
# Install
cd ~/ros2_ws
colcon build --packages-select ros2_triage --symlink-install
source install/setup.bash
pip install textual pydantic networkx pyyaml --break-system-packages

# Launch TUI (zero-config – auto-discovers everything)
ros2 run ros2_triage triage tui

# Launch with config file
ros2 run ros2_triage triage tui --config config/my_robot.yaml

# Headless preflight check (returns exit code 0=pass 1=fail)
ros2 run ros2_triage triage tui --preflight --preflight-timeout 30

# Original CLI mode still works unchanged
ros2 run ros2_triage triage check
```

End of CURSOR_PROMPT.md — paste this entire file into Cursor Composer